



Mobile and Autonomous Robots

UE23CS343BB7

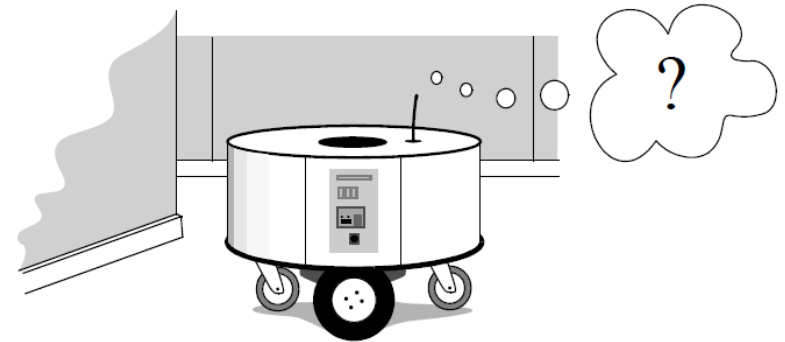
Localization

- Introduction Localization
- Challenges of Localization
- Localization-Based Navigation Vs Programmed Solutions
- To Localize or Not to Localize

Localization for Robots

Localization

- Localization is the ability of a robot to determine its own position and orientation within an environment to navigate and perform tasks effectively.
- Localization methods include the use of sensors like cameras, LIDAR, and odometry, and algorithms such as SLAM create maps of the robot's surroundings while simultaneously locating itself within that map.



Localization for Robots

Localization

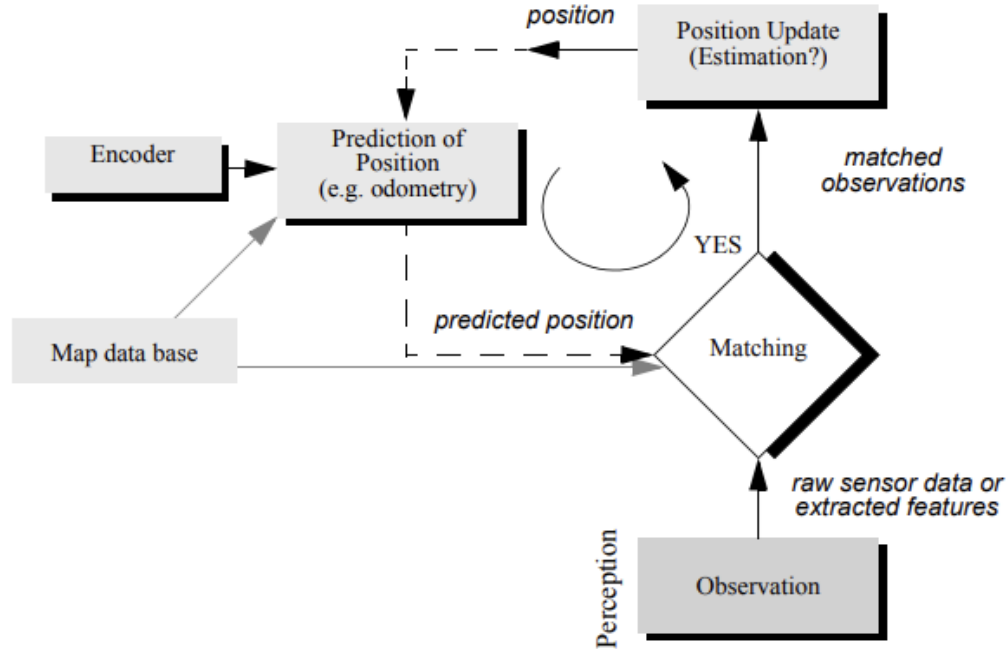
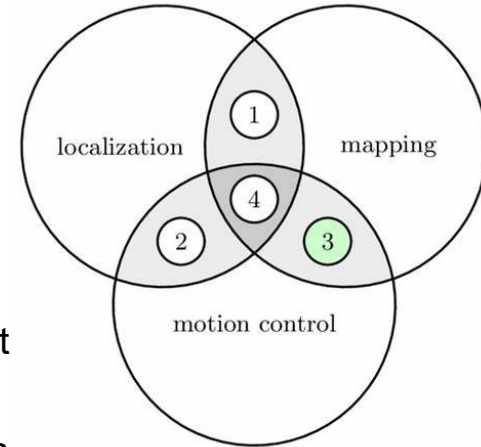


Figure 5.2
General schematic for mobile robot localization.

Localization for Robots

Challenges of Localization

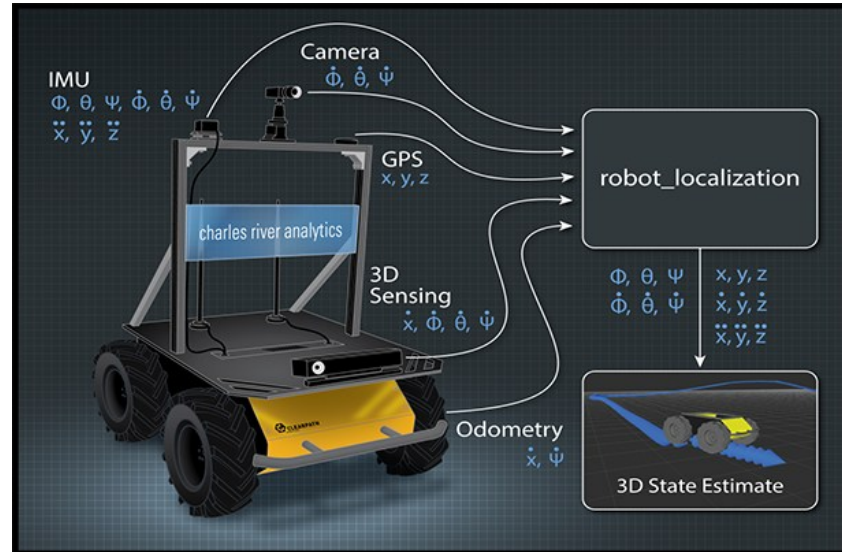
- Inaccuracies and incompleteness of these sensors and effectors pose challenges to localization such as, Noise and aliasing:
 - Sensor noise
 - Sensor aliasing
 - Effector noise
- During the cognition step a robot will select a strategy for achieving its goals.
- If it intends to reach a particular location, then localization may not be enough.
- The robot may need to acquire or build an environmental model, a map, that aids it in planning a path to the goal.
- Hence, localization means more than simply determining an absolute pose in space; it means building a map, then identifying the robot's position relative to that map.



Localization for Robots

Sensor Noise

- Sensor noise refers to:
Random variations in sensor readings even when the environment remains unchanged.
- Impact:
 - Reduces accuracy
 - Reduces information reliability
 - Makes perception difficult
- Sources:
 - Environmental changes
 - Sensor limitations
 - Electronic interference



Localization for Robots

Sensor Noise

- **Example:** CCD Camera for Indoor Navigation
 - The robot detects objects using color information.
- **Problem:** Lighting changes affect color values.
 - Example:
 - Sunlight changes due to clouds
 - Indoor illumination changes
- **Result:** The camera output appears noisy, even though the environment did not change.

Common causes of noise in vision sensors:

- Picture jitter
- Signal gain variations
- Blooming
- Blurring

These effects reduce:

- Image clarity
- Feature detection reliability

Result: Lower **information content** from visual sensors.

Localization for Robots

Sensor Noise

Handling Sensor Noise

Solutions to reduce sensor noise impact:

1. Temporal Fusion

Combine readings over time.

2. Multi-sensor Fusion

Combine data from multiple sensors:

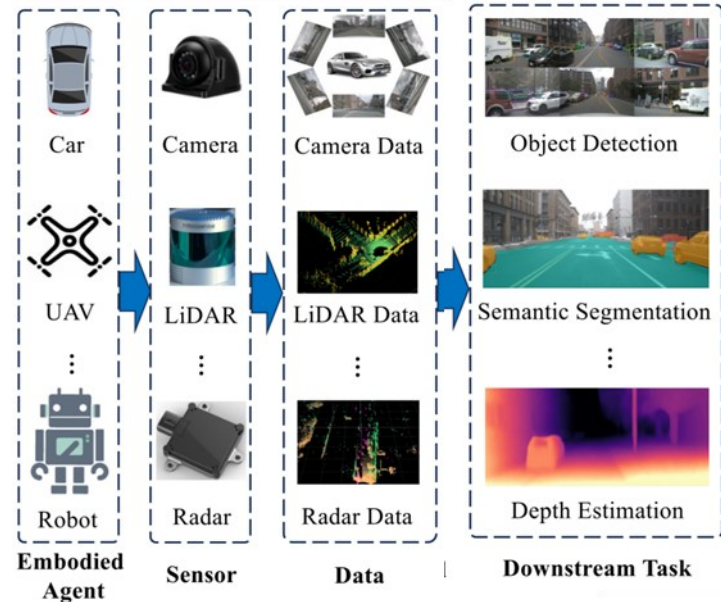
Examples:

Camera + LiDAR

Sonar + IMU

GPS + Vision

Result: Higher reliability and better localization.



Localization for Robots

Sensor Aliasing

- Occurs when different environments produce identical sensor readings.
 - In robotics:
 - Many environmental states → Same sensor output.
 - This creates ambiguity in localization.
- Human Example of Aliasing:

Humans rarely experience aliasing due to powerful vision systems.

Example scenarios:

Walking in complete darkness

Everything looks the same

Localization becomes difficult

Maze made of tall hedges

Many locations look identical

Hard to determine position

➤ **Similar problems occur in robots.**

Localization for Robots

Sensor Aliasing

Robot Sensor Aliasing Example

Consider a robot using **range sensors**.

Range sensors only measure:

- Distance to obstacle

They cannot detect:

- Color
- Texture
- Material

Example:

Robot detects an obstacle.

It cannot determine whether it is:

- A **human**
- A **cardboard box**

Both produce identical sensor readings.

Solving Sensor Aliasing:

To overcome aliasing, robots must use:

1. Multiple sensor readings over time

2. Motion-based observations

3. Environmental mapping

Example techniques:

- SLAM (Simultaneous Localization and Mapping)
- Particle filters
- Kalman filters

Localization improves as more data is collected.

Localization for Robots

Effector Noise

- Localization errors also arise from robot motion.
- Robot actions are not perfectly executed.
Reasons:
 - Wheel slip
 - Uneven terrain
 - External disturbances
- Result: The robot's actual motion differs from the intended motion.

Localization for Robots

Effector Noise: Odometry and Dead Reckoning

Robot position estimation often uses:

- **Odometry:** Uses **wheel encoder sensors**.
- **Dead Reckoning**
 - Wheel encoders
 - Heading sensors (gyro, compass)

Problem:

- Errors accumulate over time.
- Thus periodic correction is necessary.

Localization for Robots

Effector Noise: Sources of Odometry Errors

Common causes:

1. Limited sensor resolution
2. Wheel misalignment
3. Unequal wheel diameters
4. Changing wheel contact points
5. Floor irregularities
6. Wheel slipping

Some errors are:

- **Deterministic (systematic)**
- **Random (nondeterministic)**

Localization for Robots

Effector Noise: Types of Odometry Errors

Three main error types:

1. **Range Error:** Error in total distance traveled.
2. **Turn Error:** Error in robot rotation.
3. **Drift Error:** Orientation error caused by wheel differences.

Over time:

Turn and drift errors dominate localization error.

As the robot moves:

- Small angular errors increase
- Position errors grow rapidly

Even a small orientation error can produce **large position errors** after long movement.

Therefore:

Localization must be continuously corrected using sensors.

Localization for Robots

Localization-Based Navigation Versus Programmed Solutions

- Navigation system for a mobile robot requires sensors and motion control for obstacle avoidance and deliberate movement.
- Explicit localization with reference to a map may not be necessary for successful navigation.
- Behavior-based approaches advocate for sets of behaviors resulting in desired robot motion without explicit localization or path planning.

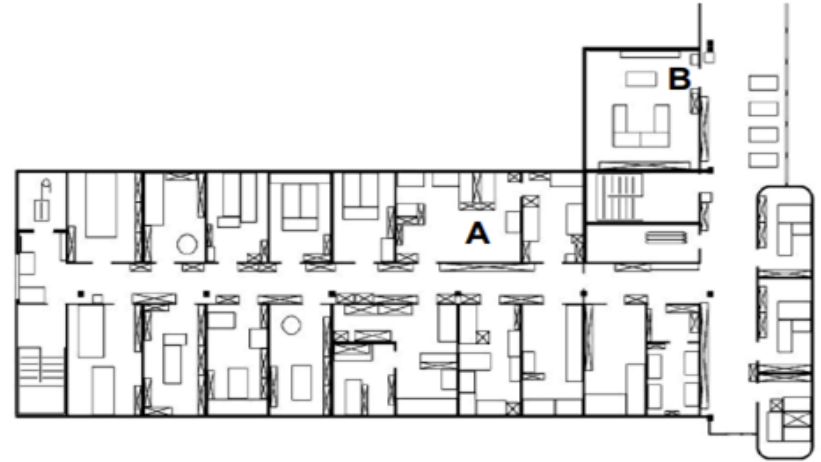


Figure 5.6
A sample environment.

Localization for Robots

Localization-Based Navigation Versus Programmed Solutions

- Example: A left-wall following behavior coupled with a room B detector triggered by a unique cue in room B could guide the robot to its destination.
- Quick to implement for specific environments but lacks scalability.
- Requires careful design of underlying procedures.
- Can suffer from instability when multiple behaviors are active simultaneously.

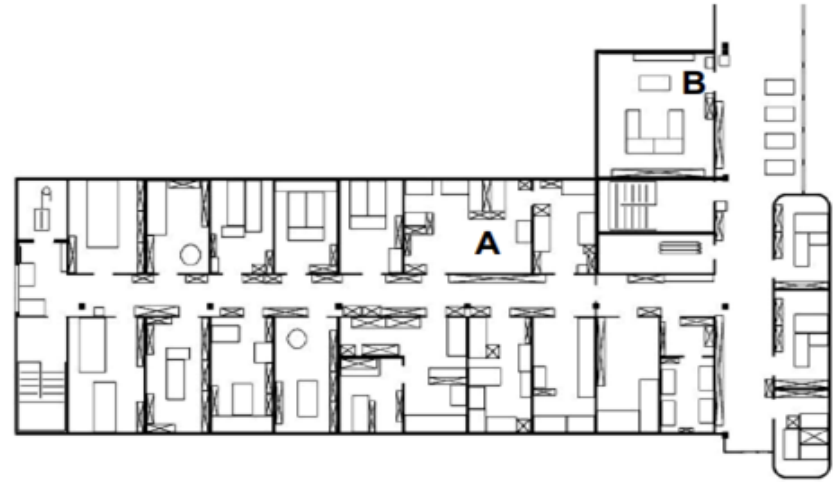


Figure 5.6
A sample environment.

Localization-Based Navigation Versus Programmed Solutions

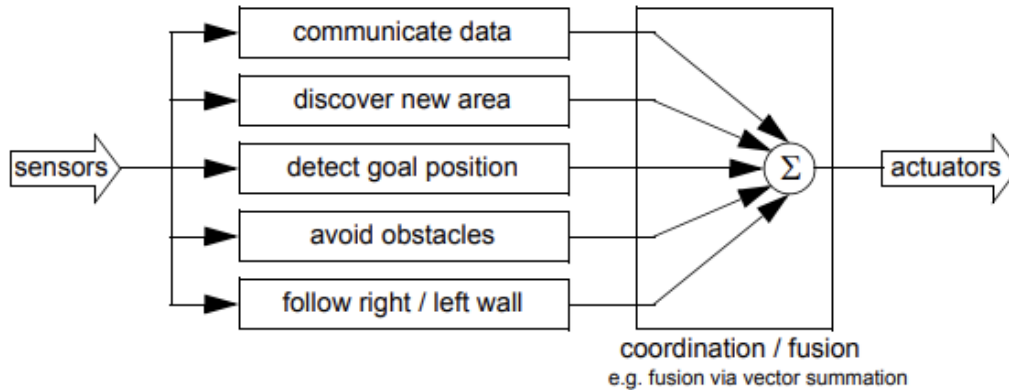


Figure 5.7

An architecture for behavior-based navigation.

Localization-Based Navigation Versus Programmed Solutions

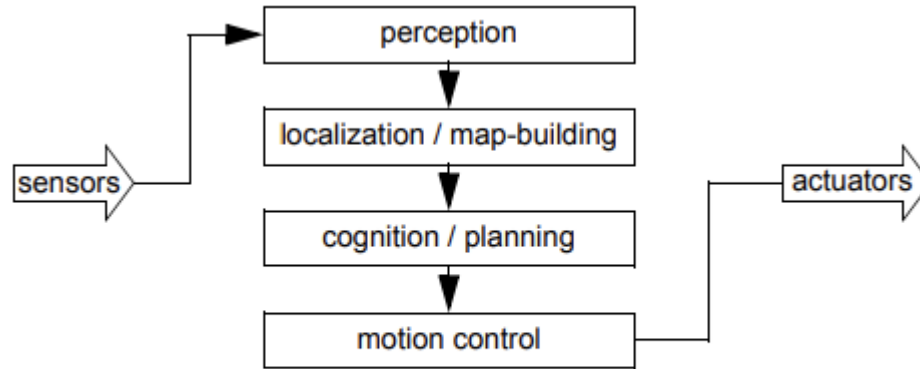


Figure 5.8

An architecture for map-based (or model-based) navigation.

Localization-Based Navigation Versus Programmed Solutions

In contrast to the **behavior-based approach**, the map-based approach includes both localization and cognition modules.

In **map-based navigation**, the robot explicitly attempts to localize by collecting sensor data, then updating some belief about its position with respect to a map of the environment.

The key advantages of the map-based approach for navigation are as follows:

- The explicit, map-based concept of position makes the system's belief about position transparently available to the human operators.
- The existence of the map itself represents a medium for communication between human and robot: the human can simply give the robot a new map if the robot goes to a new environment.

Localization-Based Navigation Versus Programmed Solutions

- The map, if created by the robot, can be used by humans as well, achieving two uses.
- Map-based approaches require more upfront development effort to create a navigating mobile robot.
- The goal is to create an architecture capable of successfully mapping and navigating various environments, spreading the design cost over time.
- The key risk is reliance on an internal representation that may diverge from reality, leading to undesirable robot behavior.



THANK YOU

Dr. Ashok Kumar Patil

Department of Computer Science and Engineering
ashokkumarp@pes.edu





Mobile and Autonomous Robots

UE23CS343BB7

Dr. Ashok Kumar Patil

Department of Computer Science and Engineering

ashokkumarp@pes.edu

TA : Gunda Sukesh



Mobile and Autonomous Robots

UE23CS343BB7

Localization

Belief Representation: Single-hypothesis and Multiple-hypothesis belief

Imagine trying to find your specific apartment in a large city. First, you need a map of the city (the environment). Second, you need to know your current address (your belief about your position). A blurry map or a confused sense of your address makes it harder to localize.

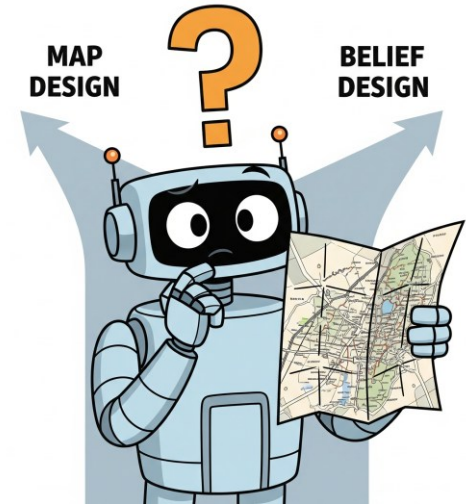
What happens to an autonomous robot in a similar situation?

For an autonomous robot, a **map is the representation of the environment**, and **its current address is its belief regarding its position**.



Belief Representations

- The core challenge in map-based localization is the issue of representation.
- Robots must model the environment (map) and their position (belief).
- **Map design** questions involve content and fidelity of the environment model.
- **Belief design** determines if the position is unique or a set of possibilities.
- Decisions along these two design axes can result in varying levels of architectural complexity, computational complexity, and overall localization accuracy.

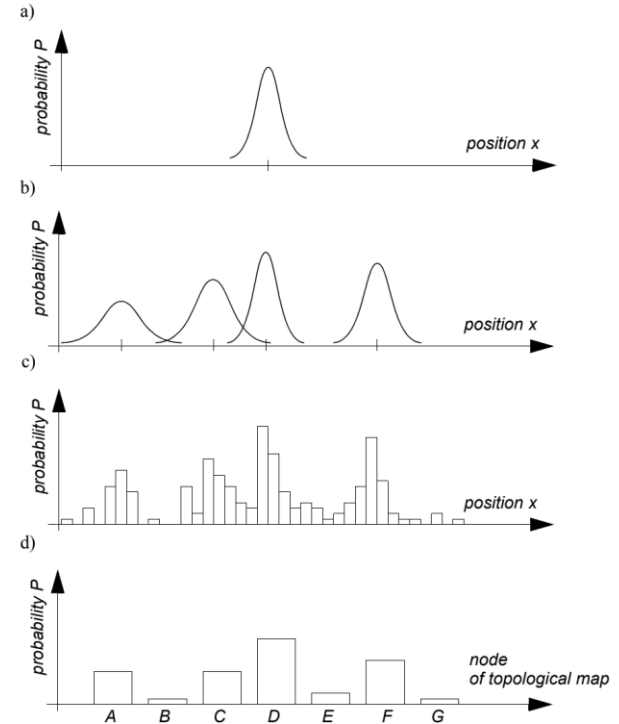


Mobile and Autonomous Robots

Belief Representations

- Belief representation systems split into single and multiple-hypothesis branches.
- Single-hypothesis systems postulate a robot's unique, single position.
- Multiple-hypothesis systems describe a robot's degree of positional uncertainty.

Belief representation regarding the robot position (1D) in continuous and discretised (tessellated) maps. (a) Continuous map with single-hypothesis belief, e.g., single Gaussian centered at a single continuous value. (b) Continuous map with multiple-hypothesis belief, e.g., multiple Gaussians centered at multiple continuous values. (c) Discretized (decomposed) grid map with probability values or all possible robot positions, e.g., Markov approach. (d) Discretized topological map with probability value for all possible nodes (topological robot positions), e.g., Markov approach.

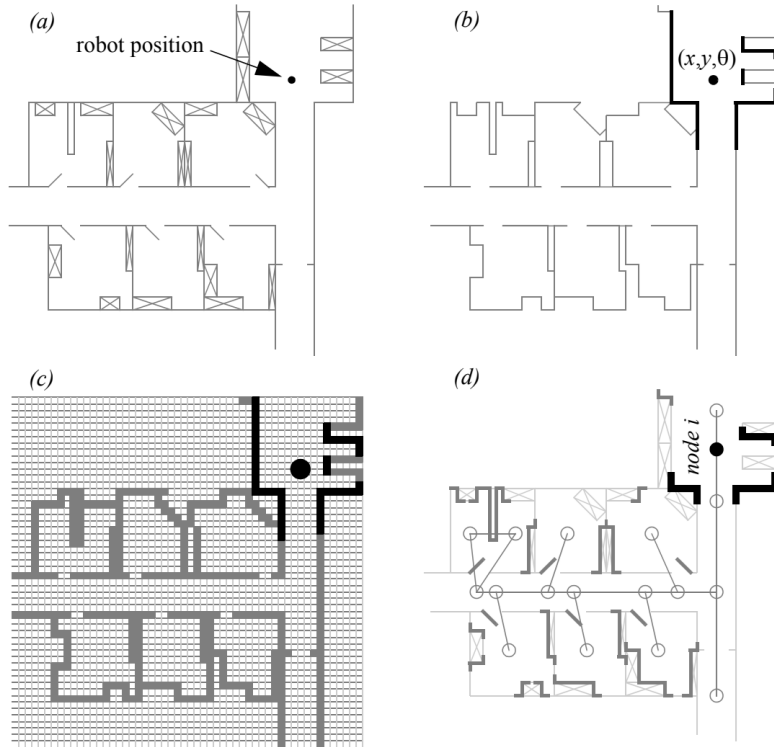


Single Hypothesis Belief

- The single-hypothesis belief representation is the most direct possible postulation of mobile robot position.
- Given some environmental map, the robot's belief about position is expressed as a single unique point on the map.
- It eliminates position ambiguity due to a single, unique belief.
- The unambiguous nature simplifies cognitive-level decision-making like path planning.
- Belief updating is facilitated as the single position is updated to a new single position.
- The main disadvantage is that motion-induced uncertainty makes forcing a single-hypothesis update challenging.

Mobile and Autonomous Robots

Single Hypothesis Belief



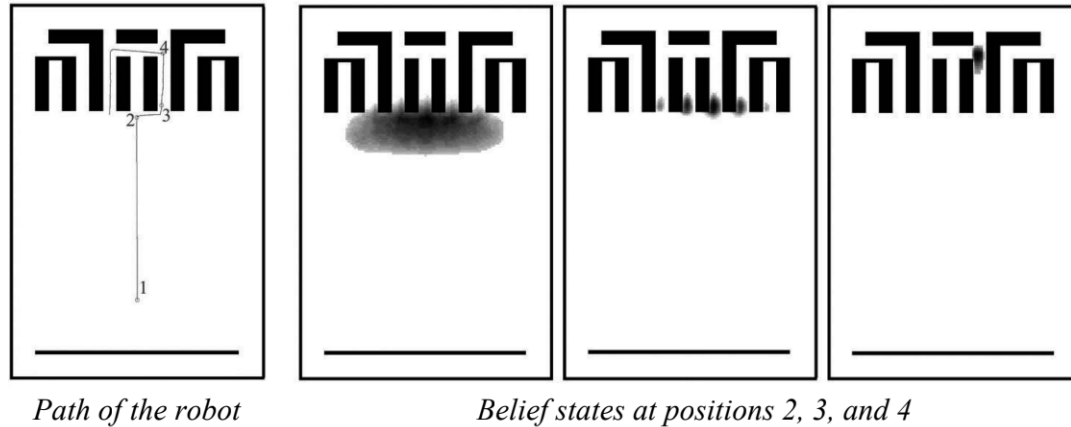
- a) Shows the actual environment layout with walls, doors, and furniture.
- b) A single point is geometrically annotated as the robot's position in a continuous 2D geometric map.
- c) The map is a discrete, tessellated one, and the position is noted at the same level of fidelity as the map cell size.
- d) The map is not geometric at all but abstract and topological. In this case, the single hypothesis of position involves identifying a single node i in the topological graph as the robot's position.

Multiple Hypothesis Belief

- In Multiple hypothesis Belief the system tracks several possible positions simultaneously. It avoids a single "best guess" to manage sensor and environmental flaws.
- The belief state is a set of possible coordinates on a 2D or 3D map.
- It explicitly manages uncertainty from noisy or partial sensor data. The robot can plan actions to actively reduce its positional uncertainty.
- Key representations include Geometric polygons and Gaussian distributions.
- The Decision-Making Dilemma complicates the robot's choice of next move. Full implementation leads to high computational complexity and state space explosion.
- Hybrid methods mitigate this by tracking clusters of high probability. Generally it is done by combining geometric maps with simplified Metric-Topological Approaches.

Mobile and Autonomous Robots

Multiple Hypothesis Belief



The belief state that is largely distributed becomes very certain after moving to position 4. The darker coloring represents higher probability.

Localization

Robot Localisation Problem

- Mobile robot navigation relies on the constant management of position uncertainty.
- When a robot begins moving from a precisely known location, it initially tracks its progress using internal odometry. However, because odometry is subject to cumulative errors, such as wheel slip or mechanical tolerances, this uncertainty grows unbounded as the robot travels.
- The processes of updating based on proprioceptive sensor values and exteroceptive sensor values are often separated logically, leading to a general process for the robot position update that comprises two steps, which are called prediction (or action) update and perception (or measurement, or correction) update.

Localization for Robots

The robot localization problem

Localization Problem and Strategy:

- Mobile robot moves in a known environment, starting from a precisely known location.
- Utilizes odometry to track motion but faces uncertainty over time (due to odometry imprecision).
- To prevent unbounded position uncertainty, the robot must localize itself relative to its environment map.
- Relies on on-board exteroceptive sensors (e.g., ultrasonic, laser, vision) to observe the environment.
- Combines odometry information with exteroceptive observations for localization.

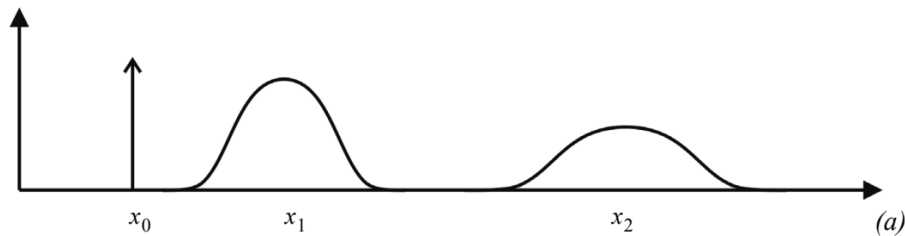
Localization Process:

- Prediction (Action) Update: Uses proprioceptive sensor values to predict robot's position.
- Perception (Measurement, Correction) Update: Incorporates exteroceptive sensor values to refine the predicted position.

Robot Localisation Problem: Prediction Phase

Prediction Phase

During the prediction (or action) update the robot uses its proprioceptive sensors to estimate its configuration; for example, the robot estimates its motion using the encoders. In this phase, the uncertainty about the robot configuration increases due to the integration of the odometric error over time. In general, the prediction update contributes to increase the uncertainty to the robot's belief about position: encoders have error, and therefore motion is somewhat nondeterministic.

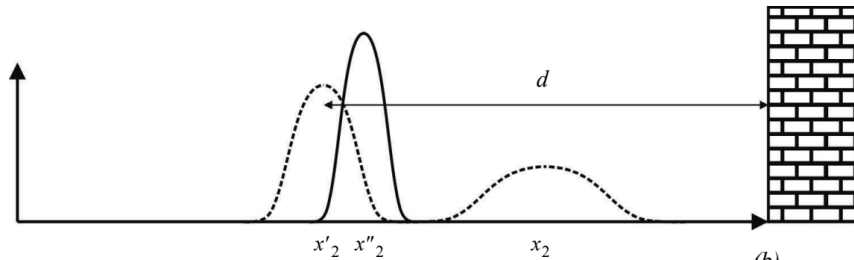


The start position x_0 is assumed to be known, and therefore the probability density function is a Dirac delta function. As the robot starts moving, its uncertainty grows due to the odometric error, which accumulates over time.

Robot Localisation Problem: Perception Phase

Perception Phase

During the perception (or measurement, or correction) update the robot uses the information from its exteroceptive sensors to correct the position estimated during the prediction phase; for example, the robot uses a rangefinder to measure its current distance from a wall and corrects accordingly the position estimated during the prediction phase. perception update generally refines the belief state (i.e., the uncertainty shrinks). Sensor measurements, when compared to the robot's environmental model, tends to provide clues regarding the robot's possible position.



The robot uses its exteroceptive sensor (e.g., a rangefinder) to measure the distance d from the right wall, and computes the position x'_2 , which is in conflict with the current position x_2 estimated in the action phase. The perception update corrects the new position to x''_2 and as consequence its uncertainty shrinks (solid line).

Localization for Robots

Prediction Update: Increases uncertainty in the robot's belief about position due to encoder errors and non-deterministic motion.

Perception Update: Refines the belief state by shrinking uncertainty through sensor measurements compared to the environmental model.

Markov Localization:

- Represents robot's belief state with any arbitrary probability density function.
- Both prediction and perception phases update the probability of every possible robot pose.

Kalman Filter Localization:

- Represents robot's belief state using a single Gaussian probability density function.
- Requires updating parameters of the Gaussian distribution to refine the belief about position relative to the map.

Localization for Robots

This fundamental difference in the representation of belief state leads to the following advantages and disadvantages of the two methods:

- **Markov localization** permits localization from any unknown position and can resolve ambiguous situations by tracking multiple disparate possible positions. However, updating the probability of all positions across the entire state space necessitates a discrete representation, like a geometric grid or a topological graph. This requirement can constrain precision and map size due to memory and computational limitations.
- **Kalman filter localization** is effective for tracking the robot from an initially known position, offering both precision and efficiency. It is suitable for continuous world representations. However, if the robot's uncertainty becomes too large, such as after a collision, and the distribution is no longer unimodal, the Kalman filter may struggle to handle multiple possible robot positions and may become irrecoverably lost.

Classification of localization problems

- **Local Localization (Position Tracking)**
 - The robot knows its initial position approximately.
 - The task is to continuously track its pose as it moves.
 - Small errors may accumulate over time (drift).
 - Example: A robot starts at a known point in a map and uses sensors (like odometry, LiDAR) to update its position.

This concept is further explored in Kalman filter localization, where the robot's belief is typically represented by a unimodal distribution, such as the normal distribution.

Classification of localization problems

- **Global localization**

- Global localization assumes the robot's initial location is unknown, allowing it to be placed anywhere in the environment without prior knowledge.
- The robot then localizes itself globally within the environment.
- Key Idea
 - The robot must determine its pose by considering all possible locations in the map.
 - Unlike local localization, there is complete initial uncertainty.
- Typically, in global localization, the robot's initial belief is represented by a uniform distribution.
- Example: Warehouse Robot
 - A mobile robot is placed inside a large warehouse.
 - The robot has a map of the warehouse, but: It does not know its starting position.

Classification of localization problems

- **Kidnapped robot problem**

- The kidnapped robot problem addresses situations where the robot is moved to a different location without its knowledge, similar to the global localization problem.
- However, the challenge arises when the robot is unaware of being relocated and erroneously believes it knows its location.
- Recovering from kidnapping is crucial for the operation of autonomous and commercial robots.
- Example:
Someone physically picks up the robot and places it elsewhere.

Other Classification of localization problems

- **Simultaneous Localization and Mapping (SLAM)**

- The robot does not have a prior map.
- It must build the map and localize itself simultaneously.
- Variants: Online SLAM (real-time) and Offline SLAM (post-processing)
- Example: Robot in an Unknown Building

It has:

- No map
- No knowledge of its position

The concept of SLAM is discussed more detail after understanding the Kalman Filter.

Other Classification of localization problems

- **Relative vs Absolute Localization**

- Relative Localization
 - Based on internal sensors (e.g., wheel encoders, IMU).
 - Uses dead reckoning.
 - Suffers from cumulative error (drift).
- Absolute Localization
 - Uses external references (e.g., GPS, landmarks)
 - Provides global position estimates.
 - More accurate but may depend on environment.

Other Classification of localization problems

- **Metric vs Topological Localization**
 - Metric Localization
 - Uses precise coordinates (x, y, θ) .
 - Requires detailed maps.
 - Topological Localization
 - Represents environment as nodes and connections.
 - Focuses on “where am I” in terms of places, not exact coordinates.



THANK YOU

Dr. Ashok Kumar Patil

Department of Computer Science and Engineering
ashokkumarp@pes.edu





Mobile and Autonomous Robots

UE22CS343BB7

Dr. Ashok Kumar Patil

Department of Computer Science and Engineering
ashokkumarp@pes.edu

TA : Gunda Sukesh



Mobile and Autonomous Robots

UE23CS343BB7

Localisation

-Probabilistic Map-Based Localisation

**-Markov Localisation and Kalman Filter
Localisation**

- Probabilistic robotics is a new approach to robotics that pays tribute to the uncertainty in robot perception and action.
- The key idea of probabilistic robotics is to represent uncertainty explicitly, using the calculus of probability theory. Put differently, instead of relying on a single “best guess” as to what might be the case in the world, probabilistic algorithms represent information by probability distributions over a whole space of possible hypotheses.
- By doing so, they can represent ambiguity and degree of belief in a mathematically sound way, enabling them to accommodate all sources of uncertainty listed above.

- Probabilistic programming in robotics allows robots to explicitly represent and manage uncertainty, making them more robust to sensor noise, imperfect data, and dynamic environments.
- It enables solutions to complex problems such as localization failures (e.g., the kidnapped robot problem) and mapping large environments where precise models or GPS are unavailable.
- However, these methods are computationally more intensive and often require approximations because representing exact probability distributions in continuous environments is infeasible.

Mobile and Autonomous Robots

Probabilistic Map-Based Localisation

START: Robot wakes up in an office. Doesn't know where.

STEP 1 - Initial Belief (before any info)

"I could be anywhere in the office"

[Belief is spread evenly across all locations]

STEP 2 - Get sensor reading

Camera sees: "I see a RED WALL"

Robot checks: "Which parts of the office have red walls?"

Answer: "The kitchen hallway has a red wall"

Updated Belief: "I'm 70% likely in the kitchen hallway"

STEP 3 - Motion with uncertainty

Command: "Move forward 1 meter"

Robot knows it might slip, so belief spreads slightly

Updated Belief: "I'm probably 1 meter forward from kitchen hallway, but might be off by $\pm 10\text{cm}$ "

STEP 4 - Get another sensor reading

Camera sees: "REFRIGERATOR"

Robot thinks: "Only the kitchen has a fridge!"

Updated Belief: "I'm now 95% sure I'm in the kitchen"

STEP 5 - More motion

Command: "Turn left 90° , move forward"

Motion uncertainty increases spread slightly again

STEP 6 - Get detailed reading

LIDAR (laser sensor) maps nearby walls

Compares to known kitchen layout

Updated Belief: "I'm at coordinate (2.3, 1.5) in the kitchen, 98% confidence"

Mobile and Autonomous Robots

Basic Concepts in Probability

Random Variable

In probabilistic robotics, quantities such as sensor measurements, controls, and the states a robot and its environment might assume are all modeled as random variables. Random variables can take on multiple values, and they do so according to specific probabilistic laws.

Probability Density Functions

In continuous spaces, random variables can take on a continuum of values and in this case we will talk about probability density functions (PDFs).

Both discrete and continuous probabilities integrate to one, therefore:

$$\sum p(x) = 1 \quad (\text{for discrete functions})$$

$$\int p(x) dx = 1 \quad (\text{for continuous functions})$$

Mobile and Autonomous Robots

Basic Concepts in Probability

Conditional Probability

The conditional probability describes the probability that the random variable X takes on the value x conditioned on the knowledge that for sure Y takes on the value y . Conditional probability is denoted with $p(x/y)$ and, if $p(y) > 0$, is defined as

$$p(x|y) = \frac{p(x,y)}{p(y)}$$

Mobile and Autonomous Robots

Basic Concepts in Probability

Gaussian Probability

$$p(x) = \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right)$$

also called normal distribution, which is commonly abbreviated with

$$p(x) = \mathcal{N}(\mu, \sigma^2)$$

Where μ and σ^2 specify the mean and the variance of the random variable. Note that in this case the random variable is a scalar. However, when x is a k -dimensional vector, we will have a multivariate normal distribution characterized by a density function of the following form:

$$p(x) = \frac{1}{(2\pi)^{k/2} \det(\Sigma)^{1/2}} \exp\left(-\frac{1}{2}(x-\mu)^T \Sigma^{-1} (x-\mu)\right),$$

where μ is the mean vector and Σ is a positive semidefinite and symmetric matrix called covariance matrix.

Mobile and Autonomous Robots

Basic Concepts in Probability

Bayes Probability

The Bayes rule relates the conditional probability $p(x/y)$ to its inverse $p(y/x)$. Under the condition that $p(y) > 0$, the Bayes rule is written as:

$$p(x|y) = \frac{p(y|x)p(x)}{p(y)}$$

As we will see that the Bayes rule is used by both Markov and Kalman-filter localisation algorithms during the measurement (Perception) update.

Mobile and Autonomous Robots

Basic Concepts in Probability

Bayes Probability Cont...

Term	Name	Meaning
	Posterior	Probability of x <i>after</i> observing y (what we want)
	Likelihood	How likely is observation x if y is true?
	Prior	Probability of x <i>before</i> observing anything
	Evidence	The total probability of observing B is the sum of all ways B could happen
		i.e $P(y) = \sum P(y x_i) \cdot P(x_i)$

Basic Concepts in Probability

Bayes Rule Example:

Scenario: A robot has a map and sees a RED WALL. Where is it?

Let's define:

• **A** = "Robot is in location x " (e.g., Kitchen, Hallway, Office)

• **B** = "Robot sees a red wall"

We want: $P(\text{Location} \mid \text{Red Wall})$

Bayes' Rule gives us:

$$P(\text{Location} \mid \text{Red Wall}) = \frac{P(\text{Red Wall} \mid \text{Location}) \cdot P(\text{Location})}{P(\text{Red Wall})}$$

Breaking it down:

1. Prior: $P(\text{Location})$

Initially, robot could be anywhere

$P(\text{Kitchen}) = 0.33$, $P(\text{Hallway}) = 0.33$, $P(\text{Office}) = 0.33$

2. Likelihood $P(\text{Red Wall} \mid \text{Location})$

• If in Kitchen: Red wall visible 95% of the time $\rightarrow P(\text{Red Wall} \mid \text{Kitchen}) = 0.95$,

• If in Hallway: Red wall visible 10% of the time $\rightarrow P(\text{Red Wall} \mid \text{Hallway}) = 0.10$,

• If in Office: Red wall visible 5% of the time $\rightarrow P(\text{Red Wall} \mid \text{Office}) = 0.05$

3. Evidence $P(\text{Red Wall})$

$$\begin{aligned} P(\text{Red Wall}) &= P(\text{Red} \mid \text{Kitchen}) \cdot P(\text{Kitchen}) + P(\text{Red} \mid \text{Hallway}) \cdot P(\text{Hallway}) + P(\text{Red} \mid \text{Office}) \cdot P(\text{Office}) \\ &= (0.95)(0.33) + (0.10)(0.33) + (0.05)(0.33) \\ &= 0.3135 + 0.033 + 0.0165 = 0.363 \end{aligned}$$

4. Posterior for Kitchen:

$$\begin{aligned} P(\text{Kitchen} \mid \text{Red Wall}) &= \frac{P(\text{Red} \mid \text{Kitchen}) \cdot P(\text{Kitchen})}{P(\text{Red})} \\ &= \frac{(0.95)(0.33)}{0.363} = \frac{0.3135}{0.363} = 0.863 \end{aligned}$$

Mobile and Autonomous Robots

Basic Concepts in Probability

Belief Distributions

In general, a robot cannot measure its true state (pose) directly, not even with GPS. What it can know is only the best estimate of its pose based on its sensors' data. Therefore, the knowledge the robot has about its state can only be inferred by the data. The “best guess” about the robot state is called “belief”.

In probabilistic robotics, beliefs are represented through conditional probability distributions, and therefore they are posterior probabilities of the state variables conditioned on the available data. If we denote the belief over a state variable x_t by $bel(x_t)$, we can write

$$bel(x_t) = \sum p(x_t | z_{1 \rightarrow t}, u_{1 \rightarrow t})$$

where the posterior $p(x_t | z_{1 \rightarrow t}, u_{1 \rightarrow t})$ represents the probability of the robot being at x_t given all its past observations $z_{1 \rightarrow t}$ and all its past control inputs $u_{1 \rightarrow t}$.

Mobile and Autonomous Robots

Probabilistic Based Localisation

Let's Recall!!

There are two different methods of probabilistic based localisation which are the Markov localization and Kalman filter localization.

Markov Localisation

In the case of Markov localization, the robot's belief state can be represented with any arbitrary probability density function. The prediction and perception phases update the probability of every possible robot pose.

Kalman Filter Localisation

Here the robot's belief state is conversely represented using a single Gaussian probability density function and thus it retains just a μ and σ parameterization of the robot's belief about position with respect to the map.

This fundamental difference in the representation of belief state leads to the following advantages and disadvantages of the two methods:

- **Markov localisation** permits localization from any unknown position and can resolve ambiguous situations by tracking multiple disparate possible positions. However, updating the probability of all positions across the entire state space necessitates a discrete representation, like a geometric grid or a topological graph. This requirement can constrain precision and map size due to memory and computational limitations.
- **Kalman filter localisation** is effective for tracking the robot from an initially known position, offering both precision and efficiency. It is suitable for continuous world representations. However, if the robot's uncertainty becomes too large, such as after a collision, and the distribution is no longer unimodal, the Kalman filter may struggle to handle multiple possible robot positions and may become irrecoverably lost.

In recent research projects, improvements are achieved or proposed by either only updating

- the state space of interest within the Markov approach
or
- by tracking multiple hypotheses with Kalman filters,
or
- by combining both methods to create a hybrid localization system.

Mobile and Autonomous Robots

UE22CS343BB7

Localisation

Markov Localisation

Markov localisation tracks the robot's belief state using an arbitrary probability density function to represent the robot's position. In practice, all known Markov localisation systems implement this generic belief representation by first tessellating the robot configuration space (x, y, ϑ) into a finite, discrete number of possible robot poses in the map. In actual applications, the number of possible poses can range from several hundred to millions of positions and orientations.

Markov localization addresses the global localization problem, the position tracking problem, and the kidnapped robot problem.

Markov Localisation Algorithm

The sensor model must calculate the probability of a specific perceptual measurement given the location of the robot and the map of the environment.

```
for all  $x_t$  do
```

$$\overline{bel}(x_t) = \int p(x_t | u_t, x_{t-1}) bel(x_{t-1}) dx_{t-1} \quad (\text{prediction update})$$

$$bel(x_t) = \eta p(z_t | x_t, M) \overline{bel}(x_t) \quad (\text{measurement update})$$

```
endfor
```

```
return  $bel(x_t)$ 
```

The critical challenge in Markov localization is the calculation of $p(z_t | x_t, M)$. The sensor model must calculate the probability of a specific perceptual measurement given the location of the robot and the map of the environment.

Three key assumptions are used to construct this sensor model:

1. Measurement Error as Gaussian Distribution

- When a range sensor detects an object, the sensor's error is assumed to follow a Gaussian distribution, centered at the correct reading.
- This distribution captures the variability and noise in sensor measurements, providing a statistical model for error estimation.

2. Nonzero Probability Across All Sensor Readings

- The sensor model must account for all possible sensor readings, even those that might deviate significantly from expected values due to noise or other factors.

- This assumption ensures the probability distribution has a nonzero value over the entire range of possible sensor readings, with the peak at the correct reading.
- A Gaussian distribution typically satisfies this requirement, with high probability at the correct reading and lower probabilities for other values.

3. Specific Failure Mode for Range Sensors

- Range sensors can experience a specific failure mode where the signal is absorbed or coherently reflected, leading to the sensor measuring its maximum range.
- To account for this, the probability distribution has a local peak at the sensor's maximum range reading.
- This aspect of the sensor model reflects the known behavior of range sensors and accommodates the inherent limitations of these devices.

LIDAR Sensor Model: Concrete Example

Setup: LIDAR with max range = 50m, actual object distance = 10m

Three Assumptions in Action

1. Gaussian Error Distribution

- Expected reading: 10m (correct distance)
- Normal readings follow: $N(10, 0.5^2)$ (mean 10m, std dev 0.5m)
- Most readings cluster around $10m \pm 0.5m$
- Captures normal sensor noise

2. Nonzero Probability Across All Readings

- Possible readings: 0m to 50m (entire sensor range)
- All values have nonzero probability
- Peak at 10m (correct reading)
- Tails extend across entire range, even to 50m

3. Failure Mode Peak at Max Range

- Small probability spike at 50m (coherent reflection failure)
- Even though object is at 10m, sensor might report 50m
- $P(\text{reading} = 50m \mid \text{actual} = 10m) = 0.05$ (5% chance of failure)

Robot's Interpretation

- **Reads 10m** → "95% sure object is at 10m" ✓
- **Reads 50m** → "Only 5% sure object is distant; 95% likely it's a coherent reflection failure" → Acts cautiously

Mobile and Autonomous Robots

UE22CS343BB7

Robot Vision and Localisation

Kalman Filter Localisation

- While Markov localisation is highly general and can represent any arbitrary probability density function, this very flexibility makes it computationally inefficient for many real-world applications.
- Instead of focusing on replicating complex density curves, robust robot localisation should prioritize the sensor fusion problem.
- Because robots utilize a wide array of heterogeneous sensors—each with its own specific failure modes—the system must find a way to reconcile these various clues to determine the robot's position accurately.

- The Kalman filter provides a powerful, efficient alternative to Markov localisation by simplifying the representation of the robot's belief state.
- It functions as an optimal recursive data-processing algorithm that incorporates all available information, regardless of its precision, to estimate the robot's current position.
- By streamlining the probability density functions of both the belief state and sensor readings, the Kalman filter achieves a level of performance that is mathematically optimal under specific conditions.

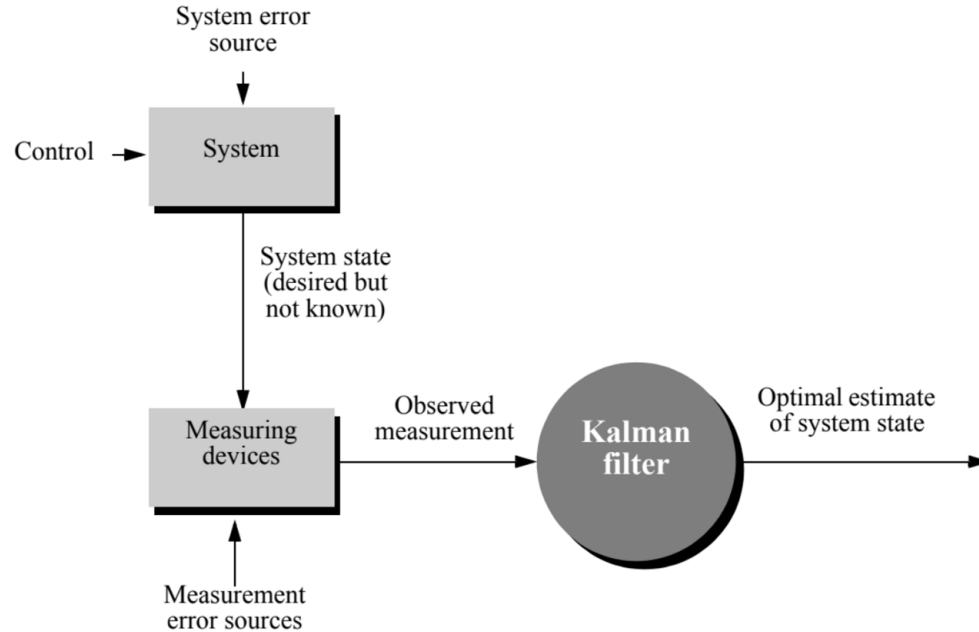


Figure depicts the general scheme of Kalman filter estimation, where a system has a control signal and system error sources as inputs. A measuring device enables measuring some system states with errors.

- In a typical application, the Kalman filter acts as a mathematical bridge between the system's control signals and its observed measurements.
- It accounts for system noise, measurement errors, and uncertainties in the dynamics models to produce an "optimal estimate" of the system state.
- Essentially, the filter fuses system knowledge (how the robot is expected to move) with sensor signals (what the robot actually sees) to minimize the impact of error sources from both the environment and the hardware.

- The mathematical optimality of the Kalman filter rests on two primary assumptions: the system must be linear, and the noise must be white Gaussian noise.
- In the field of mobile robotics, these conditions are rarely met perfectly, as most systems are inherently nonlinear.
- To address this, engineers often use the Extended Kalman Filter (EKF), which linearises the system to maintain functionality.
- While this means optimality can no longer be strictly guaranteed and the Gaussian error assumption may be technically invalid, the resulting localisation remains extremely useful and effective in practice.

Kalman Filter Localisation – Concrete Example

Scenario: Robot tracking its position using wheel encoder and noisy GPS

System Setup

State: Robot position x (1D for simplicity)

Measurements:

Wheel encoder: "I moved 1m forward"

GPS: "You're at position 10.2m" (noisy)

Assumptions (Linear & Gaussian):

Motion model is linear: $x_t = x_{t-1} + u_t$ (position + movement)

Measurement noise is Gaussian: $\mathcal{N}(0, 0.5^2)$

Process noise is Gaussian: $\mathcal{N}(0, 0.1^2)$

Kalman Filter Localisation – Concrete Example

Step-by-Step Kalman Filter

Initial state:

Prior estimate: $\hat{x}_0 = 0\text{m}$

Uncertainty: $P_0 = 1\text{m}^2$ (high uncertainty)

Time t=1:

Predict (wheel encoder: moved 1m)

Predicted position: $x_1 = 0 + 1 = 1\text{m}$

Predicted uncertainty: $P_1 = 1 + 0.1^2 = 1.01\text{m}^2$

Measure (GPS reads: 1.1m)

Measurement: $z_1 = 1.1\text{m}$

Update (Kalman gain balances encoder vs GPS)

$$K = \frac{P_1}{P_1 + 0.5^2} = \frac{1.01}{1.26} = 0.80$$

Updated estimate: $\hat{x}_1 = 1 + 0.80(1.1 - 1) = 1.08\text{m}$

Updated uncertainty: $P_1 = (1 - 0.80) \times 1.01 = 0.20\text{m}^2$

Time t=2: Repeat

Predict (moved 1m again)

Position: $x_2 = 1.08 + 1 = 2.08\text{m}$

Uncertainty: $P_2 = 0.20 + 0.01 = 0.21\text{m}^2$

Measure (GPS reads: 2.3m)

Update

$$K = \frac{0.21}{0.21 + 0.25} = 0.46$$

$\hat{x}_2 = 2.08 + 0.46(2.3 - 2.08) = 2.18\text{m}$

$P_2 = 0.11\text{m}^2$ (uncertainty shrinks)

Kalman Filter Localisation – Concrete Example

Step	Estimate	Uncertainty
Start	0 m	1.00 m ²
t=1	1.08 m	0.20 m ² ↓
t=2	2.18 m	0.11 m ² ↓
t=3+	Converges	→ 0

Key Insight

What happens:

- Wheel encoder is predictable but drifts
- GPS is noisy but corrects drift
- Kalman Filter blends both optimally
- Uncertainty decreases over time → Robot becomes more confident

Result: Despite noise, robot accurately tracks position



THANK YOU

Dr. Ashok Kumar Patil

Department of Computer Science and Engineering
ashokkumarp@pes.edu





Mobile and Autonomous Robots

UE22CS343BB7

Dr. Ashok Kumar Patil

Department of Computer Science and Engineering
ashokkumarp@pes.edu

TA : Gunda Sukesh



Mobile and Autonomous Robots

UE23CS343BB7

Localisation

SLAM: The Simultaneous Localization and Mapping Problem

- Traditional mapping is manual, costly, and time-consuming. These maps are static and struggle with dynamic changes, such as rearranged furniture. Because they rely on human perception, they lack the flexibility needed for real-world robot navigation.
- The alternative to handmade map building is “automatic map building.”
- True autonomy requires automatic map building, where a robot uses its own sensors to perceive and document its surroundings.
- This removes human dependency and allows the robot to adapt to environmental changes in real-time.
- By mastering place recognition and environmental modification, robots can now explore and map unknown spaces independently. Achieving this self-sufficient mapping is the final step in making a mobile robot truly autonomous.

- The recent advances in both robotics and computer vision have made this goal somewhat achieved.
- However, a mobile robot's sensors have only a limited range, and so the robot must physically explore its environment to build such a map.
- So, the robot must not only create a map, but it must also do this while moving and localizing to explore the environment. In the robotics community, this is often called the **Simultaneous Localisation and Mapping (SLAM) problem**.

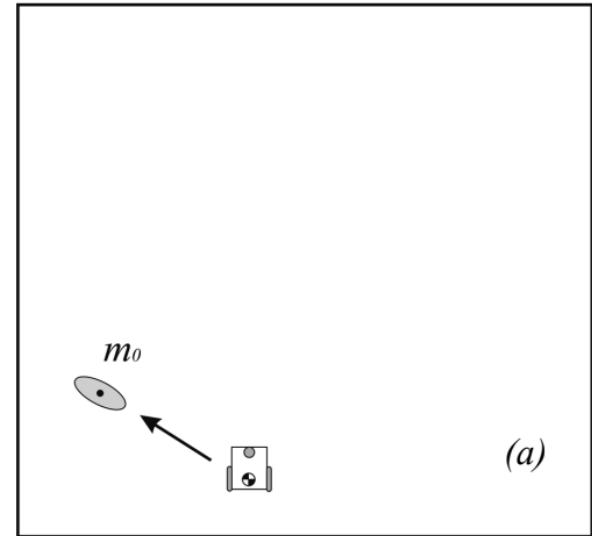
SLAM: The Simultaneous Localisation and Mapping Problem

- Localisation is the problem of estimating the robot position (and therefore its path) given a known map of the environment.
- Conversely, mapping is the construction of the map of the environment knowing the true path of the robot.
- The aim of SLAM is to recover both the robot path and the environment map using only the data gathered by its proprioceptive and exteroceptive sensors.
- These data are typically the robot displacement estimated from the odometry and features (e.g., corners, lines, planes) extracted from laser, ultrasonic, or camera images.
- SLAM is difficult because both the estimated path and the extracted features are corrupted by noise.

SLAM: The Simultaneous Localisation and Mapping Problem

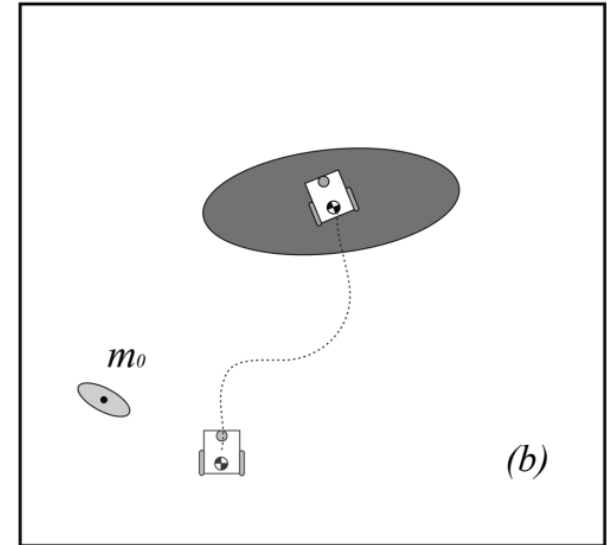
Let us assume that the robot uncertainty at its initial location is zero. From this position, the robot observes a feature which is mapped with an uncertainty related to the exteroceptive sensor error model.

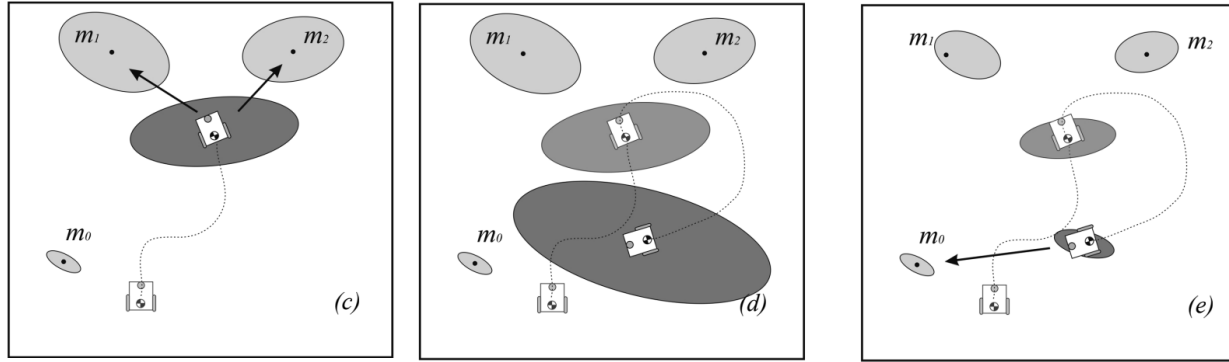
- (a) As the robot moves, its pose uncertainty increases under the effect of the errors introduced by the odometry.



SLAM: The Simultaneous Localisation and Mapping Problem

(b) At this point, the robot observes two features and maps them with an uncertainty that results from the combination of the measurement error with the robot pose uncertainty





(c) From this, we can notice that the map becomes correlated with the robot position estimate. Similarly, if the robot updates its position based on an observation of an imprecisely known feature in the map, the resulting position estimate becomes correlated with the feature location estimate. In order to reduce its uncertainty, the robot must observe features whose location is relatively well known. These features (d) can be landmarks that the robot has already observed before. In this case, the observation (e) is called loop closure detection. When a loop closure is detected, the robot pose uncertainty shrinks.

SLAM: The Simultaneous Localisation and Mapping Problem

- At the same time, the map is updated and the uncertainty of other observed features and all previous robot poses also reduce.
- The general problem of map-building is thus an example of the chicken-and-egg problem. For localisation the robot needs to know where the features are, whereas for map-building the robot needs to know where it is on the map.



THANK YOU

Dr. Ashok Kumar Patil

Department of Computer Science and Engineering
ashokkumarp@pes.edu



Mobile and Autonomous Robots

UE22CS343BB7

Dr. Ashok Kumar Patil

Department of Computer Science and Engineering
ashokkumarp@pes.edu

TA : Gunda Sukesh

Mobile and Autonomous Robots

UE23CS343BB7

Localisation

- Extended Kalman Filter (EKF) SLAM,
- Particle filter SLAM,
- Open challenges in SLAM

- EKF-SLAM (Extended Kalman Filter SLAM) is a technique used in mobile robotics to simultaneously build a map of an unknown environment and localise the robot within it.
- It addresses the challenge of real-time processing and handling uncertainties that arise from sensor noise and motion inaccuracies.
- The algorithm works by maintaining a state vector that includes both the robot's position and the locations of detected landmarks.
- It also maintains a joint covariance matrix which represents the uncertainties and correlations between the robot's pose and all mapped feature locations, capturing how errors in one estimate affect the others.

- During the update step, sensor measurements from devices like LIDAR or cameras are used to correct the predicted state, minimizing errors through the Kalman Filter equations.
- One major advantage of EKF-SLAM is its ability to perform real-time localization and continuous map updates while handling uncertainty.
- Despite its efficiency in low-dimensional environments, EKF-SLAM is computationally expensive, with complexity increasing quadratically with the number of landmarks.

Challenges

- EKF-SLAM combines the robot pose and feature locations into a joint state and updates both the state and joint covariance matrix at every iteration.
- The computation in EKF-SLAM grows quadratically with the number of features, making it computationally expensive for large environments.
- Efficient real-time implementations have been developed by dividing the map into smaller submaps and updating their covariances separately.
- Linearisation errors occur due to the use of Jacobians for motion and measurement updates, which can lead to inconsistencies or divergence in highly non-linear scenarios.

Mobile and Autonomous Robots

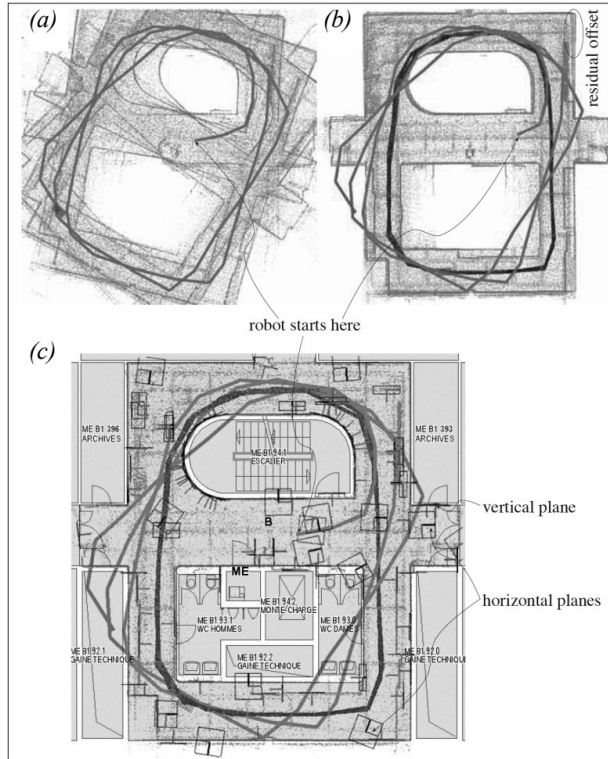
Extended Kalman Filter SLAM

Challenges

- EKF-SLAM is prone to incorrect data association, especially during loop closure, where matching current observations with previously mapped features can fail.
- Cameras and advanced feature detectors (like SIFT) significantly improve loop closure detection and reduce data association errors compared to 2D laser rangefinders.
- Maintaining correlations between features is crucial, as more observations increase correlation and improve accuracy.
- Over time, feature correlations become stronger, leading to a more accurate map, where relative errors decrease and the map covariance matrix converges toward zero.

Mobile and Autonomous Robots

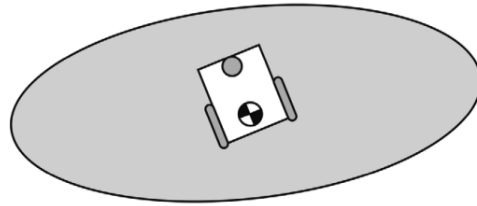
Extended Kalman Filter SLAM



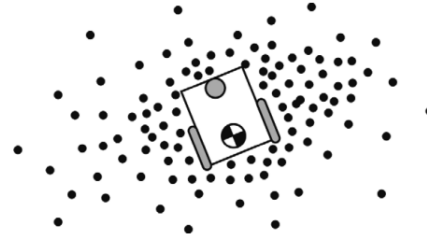
EKF SLAM using 3D laser scanner.

- (a) The robot starts at the center and makes three rounds. Aligned 3D scans using odometry only leading to an inconsistent map.
- (b) Aligned 3D scans after using scan matching. The accumulated odometry error can be drastically reduced, but a small residual error remains (see offset).
- (c) The result of the EKF SLAM. The map built has been superimposed on a building plan for visual comparison.

- Particle filter SLAM is a technique used in robotics and autonomous systems to estimate both the pose (position and orientation) of a robot and the map of its environment simultaneously.
- The term particle filter is born from the fact that it represents the robot belief distribution not in a parametric form (like a Gaussian) but rather as a set of samples (i.e., particles) drawn randomly from this distribution.
- This concept is pictorially illustrated in figure. The power of this representation is in its ability to model any sort of distribution (e.g., non-Gaussian) and also nonlinear transformation.



(a)



(b)

The standard EKF SLAM represent the probability distribution of the robot location is a parametric form, which is a two-dimensional Gaussian

- (a) Conversely, particle filter SLAM represent this the probability distribution as a set of particles drawn randomly from the parametric distribution
- (b) For the specific example of a Gaussian distribution, (b) the density of particles is higher toward the center of the Gaussian and decreases with the distance.

- **Bayesian Filter:** Particle filter SLAM is a Bayesian filtering approach, which means it uses probabilistic methods to estimate the robot's pose and map. It maintains a probability distribution over possible poses and maps based on sensor measurements and motion updates.
- **Particle Representation:** In particle filter SLAM, the belief distribution is represented by a set of particles, where each particle represents a hypothesis of the robot's pose and the map. These particles are sampled from the posterior distribution and are updated over time based on sensor measurements and motion commands.

- **Localisation and Mapping:** Particle filter SLAM addresses both the localisation problem (estimating the robot's pose relative to the map) and the mapping problem (building a map of the environment). It does this by maintaining a joint probability distribution over the robot's pose and the map.
- **Resampling:** As the robot moves and gathers sensor measurements, the particles are updated using a process known as importance sampling. During this process, particles with higher likelihoods are more likely to be sampled, while particles with lower likelihoods are discarded. This helps focus computational resources on regions of the state space with higher probability.

- **Data Association:** Particle filter SLAM often incorporates techniques for data association to associate sensor measurements with features in the environment. This helps in building the map and updating the robot's pose accurately.
- **Challenges:** Particle filter SLAM faces challenges such as the curse of dimensionality, where the number of particles required to represent the belief distribution accurately grows exponentially with the dimensionality of the state space. Additionally, handling uncertainties in sensor measurements and motion models is critical for accurate estimation.

- **Real-Time Operation:** Despite its computational complexity, particle filter SLAM can be implemented in real-time systems. Techniques such as resampling and approximate methods like FastSLAM help in maintaining efficiency while providing accurate estimates.
- **Applications:** Particle filter SLAM has applications in various domains, including robotics, autonomous vehicles, augmented reality, and virtual reality. It enables robots and autonomous systems to navigate and interact with their environments autonomously while building accurate maps for localization and navigation.

Mobile and Autonomous Robots

Open Challenges in SLAM

- **Dynamic Environments:** Moving objects (vehicles, animals, humans) violate the static map assumption, requiring algorithms to either treat them as outliers or predict their motion.
- **Multi-Robot Mapping:** The difficulty of combining individual readings from a team of multiple robots into a single, cohesive global map.
- **Data Association and Loop Closure:** High sensitivity to matching errors when a robot returns to a previously visited location, which can lead to map failure.
- **Sensor Feature Limitations:** 2D laser rangefinders struggle to identify unique, distinctive features in point clouds compared to the rich information provided by cameras.
- **Monocular Scale Ambiguity:** Single-camera systems provide only bearing information, requiring external data (GPS, IMU, or known object sizes) to recover the absolute scale.



THANK YOU

Dr. Ashok Kumar Patil

Department of Computer Science and Engineering
ashokkumarp@pes.edu